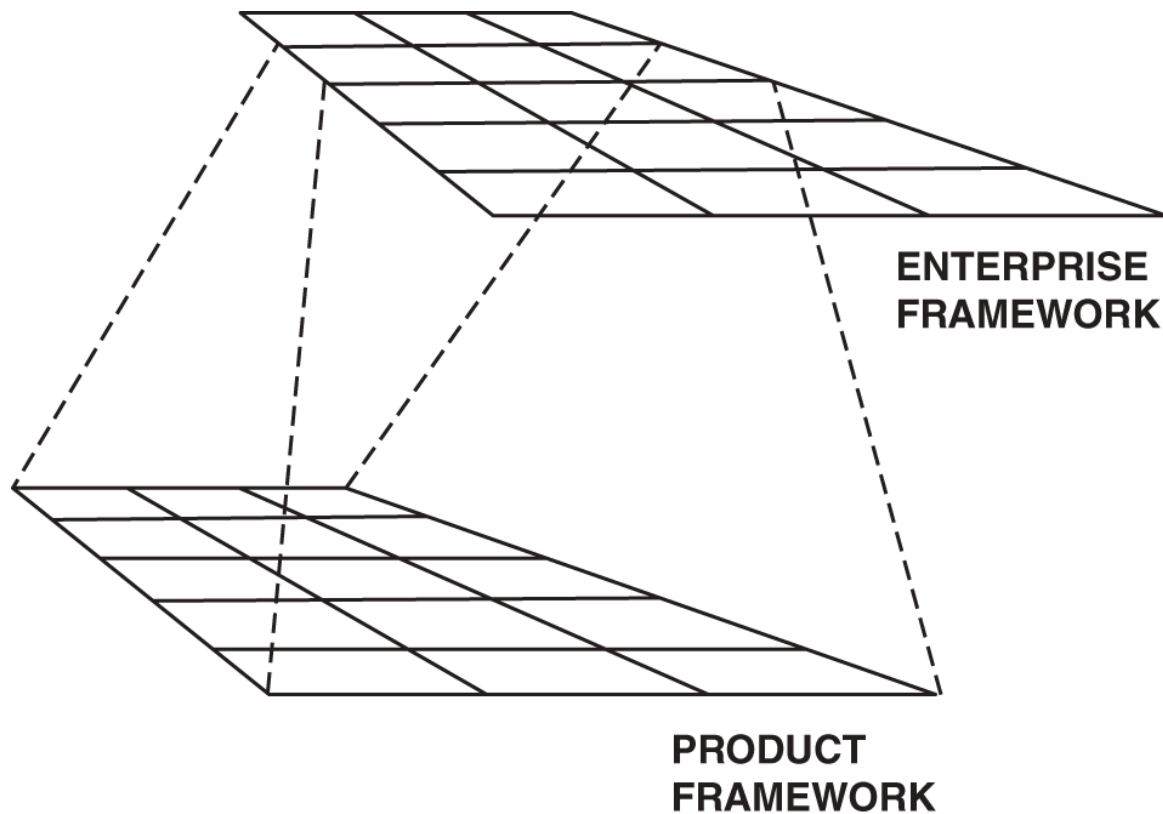


and Methodology (GERAM) (Bernus, 1999) can be used to establish and maintain a set of capabilities, services, and resources that support the architecture process. The enablement activities include:

- Analysis of context in the organization where the architecture activities can take place.
- Definition of the main principles and the overall organization where the processes, methods, roles, and technologies can be used for architecting.
- Implementation of these high-level principles with methodologies provided by frameworks like TOGAF for IT domain or NAF and DoDAF for defense domains. This implementation comprises development of a metamodel to capture the terminology and a collection of architecture development methods, standards library, architecture repository and registry, and architecture capability. Architecture capability includes skills and governance logic.
- Reference documents like Evans (2014) help assess the architecture context and environment with regard to the architecting styles of the enterprise programs and projects.

*Architecture Governance* Related concepts are objective, goal, strategy, policy, directive, roadmap, life cycle stage, and statement of work. Per Sowa and Zachman (1992), as illustrated in [Figure 3.16](#), two levels of architecture frameworks should be established to ensure consistency of products and systems regarding the enterprise strategy. Within most contexts, there exists a need to consider the architecture of multiple entities, each with its own life cycle, and correspondingly a framework, that helps describe or model that entity. In addition, architecting must consider that the life cycles of these entities are interrelated, often in a recursive manner (by one entity contributing to some or all activities in the life cycle of another), and that these activities may have to be synchronized (e.g., for complexity reduction purposes, to achieve or to maintain selected system quality characteristics).



**FIGURE 3.16** Enterprise and product frameworks. From Sowa and Zachman (1992). Used with permission. All other rights reserved.

Frameworks like NAF and TOGAF consider two levels of architecture governance:

- At *enterprise* level, in accordance with enterprise objectives, goals, strategy, roadmap, policies and directives.
- At *project* level, with regard to internal or external contracts of the projects of produced architecture(s) along the whole life cycle of the entity of interest related to the architecture(s).

*Architecture Management* Architecture management implements the governance directives in the frame of the project contract. Frameworks like NAF explain that the architecting effort should be defined in a management plan. This plan should also include the management of the work products throughout their whole life cycle. The management activities coordinate the architecting effort and report to the governance level with rationales about the application of the governance directives and the agreement.

*Architecture Description* Architecture Description, as defined by ISO/IEC/IEEE 42010 (2022), is accomplished with *Architecture Conceptualization* and *Architecture Elaboration*, as defined by ISO/IEC/IEEE 42020 (2019). These architecting activities are performed as planned by the management plan covering the architecture effort. Frameworks like Zachman explain how architecture viewpoints characterize the problem and represent the solution with regard to the stakeholders' concerns, possibly structured as architecture aspects and stakeholders' perspectives. These viewpoints can be defined in architecture frameworks or developed within the project for the benefit of the enterprise. ISO/IEC/IEEE 42010 defines *views* and *viewpoints* as they apply to the architecture description. Annex F of ISO/IEC/IEEE 42010 includes tables of requirements compliance for Architecture Description Frameworks. Development of project-specific viewpoint specifications needs to be justified because they imply additional effort for architecture description, evaluation, management, and usage of the work products.

Frameworks like NAF and TOGAF include a methodology to develop the architecture views governed by the viewpoints. As far as possible, this development should be based on patterns and standards already proven in the business domain where the entity of interest resides. Formalisms, model kind specifications, and modeling languages are typically defined in AFs.

*Architecture Evaluation* As defined by ISO/IEC/IEEE 42030 (2019), the evaluation activities determine the extent to which one or more architectures meet their objectives, address stakeholder concerns, and meet relevant requirements. These activities are performed as planned by the management plan covering the architecture effort. Frameworks like Architecture Tradeoff Analysis Method (ATAM) (CMU/SEI, 2000) and Method Framework for Engineering System Architectures (MFESA) (Firesmith, et al. 2008) allow performing architecture evaluation in three steps:

- Definition of the objectives and evaluation criteria agreed by the stakeholders to cover their concerns.

- Definition or development of a method to cover the activities normally structured in analysis, assessment, and evaluation tasks.
- Analysis of the architecture concepts and properties, assessment of the value and utility for the stakeholders, and formulation of findings and recommendations in evaluation reports.

### 3.2.6 Patterns

#### *Introduction to Patterns*

The scientific disciplines, whether concerned with phenomena at a molecular, global, or astronomical scale, are based upon discovery and effective modeling of *patterns*. Patterns are recurrences—repeated regularities observed across time, space, or other dimensions. Patterns lie at the heart of physical sciences and the related engineering disciplines, as laws of nature whose mathematical representation and engineering exploitation have transformed the nature and possibilities of human life. In SE, recurring patterns are observable in engineered system requirements, solution architectures, stakeholder value, missions, fitness and trade spaces, parametric couplings, failure modes and risks, markets, system phenomena, principles, and the socio-technical systems of engineering and life cycle management. For example, there are patterns for requirements for refrigerators, patterns for design of coolant compressors, patterns for refrigerant failures, and patterns for maintaining refrigerators. Patterns are visible for products developed for commercial markets and systems engineered under defense contracts, as well as for the socio-technical systems that produce them, such as methodology patterns for eliciting and validating requirements. Whether the patterns are only implicitly and informally recognized and used, or explicit and formal, they can be found across the System of Innovation Pattern shown in [Figure 1.6](#), where they are the basis of group learning. Explicitly modeled patterns help us surface and more efficiently share (learn, teach, practice) what earlier generations of SE practitioners treated as expertise and intuition only obtainable over decades of personal practice. As in the physical sciences, engineering patterns of all kinds are also subject to issues of credibility, validity,

applicability, and trust as a basis for decision-making and action. Patterns are not “one size fits all”, but instead have both fixed (recurring) and variable (parameterized) aspects, distilled by abstraction across individual instances. Depending on how they are recognized, represented, managed, and applied, patterns may be *informal* or *formal*.

### ***Informal Patterns***

The most informally described patterns are those implicit in the expertise or judgement of individual practitioners and teams (as in tribal knowledge), when subject matter experts recognize new occurrences of past experiences. Examples are Jean’s expertise in packaging systems, or Jose’s expertise in risk assessment. Because of the high value of this experience and interest in making it available to others, historical efforts have been made to explicitly capture and record such patterns, even in informal form, so that they can be transmitted to others. SE *principles* and *heuristics*, often captured as informal prose, illustrate such explicit but informal patterns (see Sections 1.4.3 and 1.4.4). The informal but explicit prose representation of engineering patterns has created popular followings in civil and software engineering communities of practice (Alexander, et al., 1977) (Gamma, et al., 1995). These patterns typically include a prose template description of a problem and an informal description of a design pattern suited to such a problem. Examples of these explicit, informal, but effective patterns include building structural patterns and city layout patterns (in civil architecture patterns), as well as sorting algorithms and graphic user interface designs (in software design patterns). SE practitioners and leaders should not underestimate the value of explicit informal patterns for transmitting knowledge.

### ***Formal Patterns***

The sciences’ transition from informal prose to formal models powered much of the Science, Technology, Engineering, and Math (STEM) revolution’s transformative impact, where model-based representations of patterns are the heart of the related physical sciences. These models have

also enabled several generations of powerful automation tools for design, simulation, and production across the engineering disciplines, and more recently this is also impacting SE.

The practice of SE has increased use of explicit formal system descriptive models as central to SE methods, described as Model-Based Systems Engineering (MBSE) (see [Section 4.2.1](#)). This also enables the shift to formal model-based representation of patterns and their application in SE, because patterns based in models can be readily transformed (including automated assistance) into configured models specific to an application or project. Likewise, such patterns can be used in automated conformance-checking of other models. Provided the credibility of the patterns for the uses intended is managed, this not only shortens time to a trustable specific model, it also helps shift the language and perspective of multiple systems practitioners and teams into common semantic frameworks specific to a domain or specialty, for improved compatibility and interoperability. For example, do designers of tractors and trailers have a common perspective on the interactions between these engineered products? Can their work be readily checked for consistency? These issues have major impacts on SE effectiveness and productivity.

Formal patterns, particularly when model-based, appear under different names and “flavors” across SE practice and this handbook. Among these are ontologies, Architectural Frameworks, schemas, and Product Line Engineering (PLE) datasets. For more on these, refer to INCOSE S\*Patterns Primer (2022) and the other sections of this handbook. Formal patterns also include general and domain-specific system modeling languages.

The power of models in the STEM revolution was not simply that they reflected agreements across groups (as in standards), but also agreements with observed natural phenomena, reduced to simplest form in the patterns of physical laws. These phenomena-based patterns continue to provide the theoretical basis for the individual engineering disciplines, as well as for the foundations of SE (Schindel, 2016, 2020). The central question they address is: What is the *smallest* system model content

necessary to represent a SoI, across its life cycle, for purposes of engineering and life cycle management (Schindel, 2011)? This question has practical implications, but is also rooted in the foundations of SE:

- The *practitioner* has an interest in keeping things as simple as possible, but not simpler. “Too large” a model implies the burden of more information than is needed, including redundancies which often include inconsistencies. “Too small” a model implies that information needed during the life cycle is missing.
- *Foundations of an engineering discipline* include representing recurring phenomena fundamental to its corresponding science. The smallest set of elements generating a discipline identifies its foundations (e.g., Newton’s Laws generating Mechanics, Maxwells’ Equations generating Electrical Science). A definition of a system’s mathematical complexity is the size of its smallest generating representation (Li and Vitanyi, 2009).

The *Systematica Metamodel* (*S\*Metamodel*) is a formal pattern describing a neutral (independent of specific modeling languages or tools) answer to the above “smallest model” question, mapped into contemporary model tooling and languages, such as SysML, simulations, or modeling frameworks. An *S\*Model* is any model, expressed in any modeling language or tooling, that is mapped to the reference *S\*Metamodel*. The *S\*Metamodel* spans disciplines, tooling, and languages, and is rooted in the phenomena-based models of the physical sciences.

Modern word processing tools are powerful, but varying writer composition skills and practices allow authoring that may produce valuable literature or faulty descriptions and broken semantics. Similarly, observed methods of use of contemporary modeling languages and automated tools allow the generation of system models that are both too small (are missing important elements) and too large (contain undetected redundancies and contradictions) at the same time. Fortunately for formal models, the history of the physical sciences provides patterns about the nature of phenomena and their models, and these can guide the users of contemporary tools and languages to more effective models than bare languages and tools alone.

Accordingly, the S\*Metamodel provides that guidance in any language or tooling into which it is mapped. Three examples from the S\*Metamodel are:

- *All behavior is interaction-based*: Physics has made it clear that there is no “naked” behavior in the absence of interactions, although system modelers sometimes create models that incorrectly assert otherwise. Interactions are the heart of system phenomena, emergence, SE, and S\*Models. Failure to understand and represent interactions leads to well-known engineering problems such as overlooking the impact of “external” actor behavior on the performance of an in-service engineered system (Schindel, 2013, 2016).
- *Requirement statements are transfer functions*: Models can help make it clear that requirement statements are not simply prose, but always represent input-output relationships parameterized by state variables. S\*Models make this clear and enable improved auditing for problematic or overlooked requirements (Schindel, 2005).
- Stakeholder value trade space, failure effects in risk analysis, and configurability of product line families are all manifestations of the same variables: These are frequently treated as relatively independent specialties and dimensions, greatly over-complicating system representation and understanding, when that apparent dimensionality can be substantially reduced by S\*Models (Schindel, 2010).

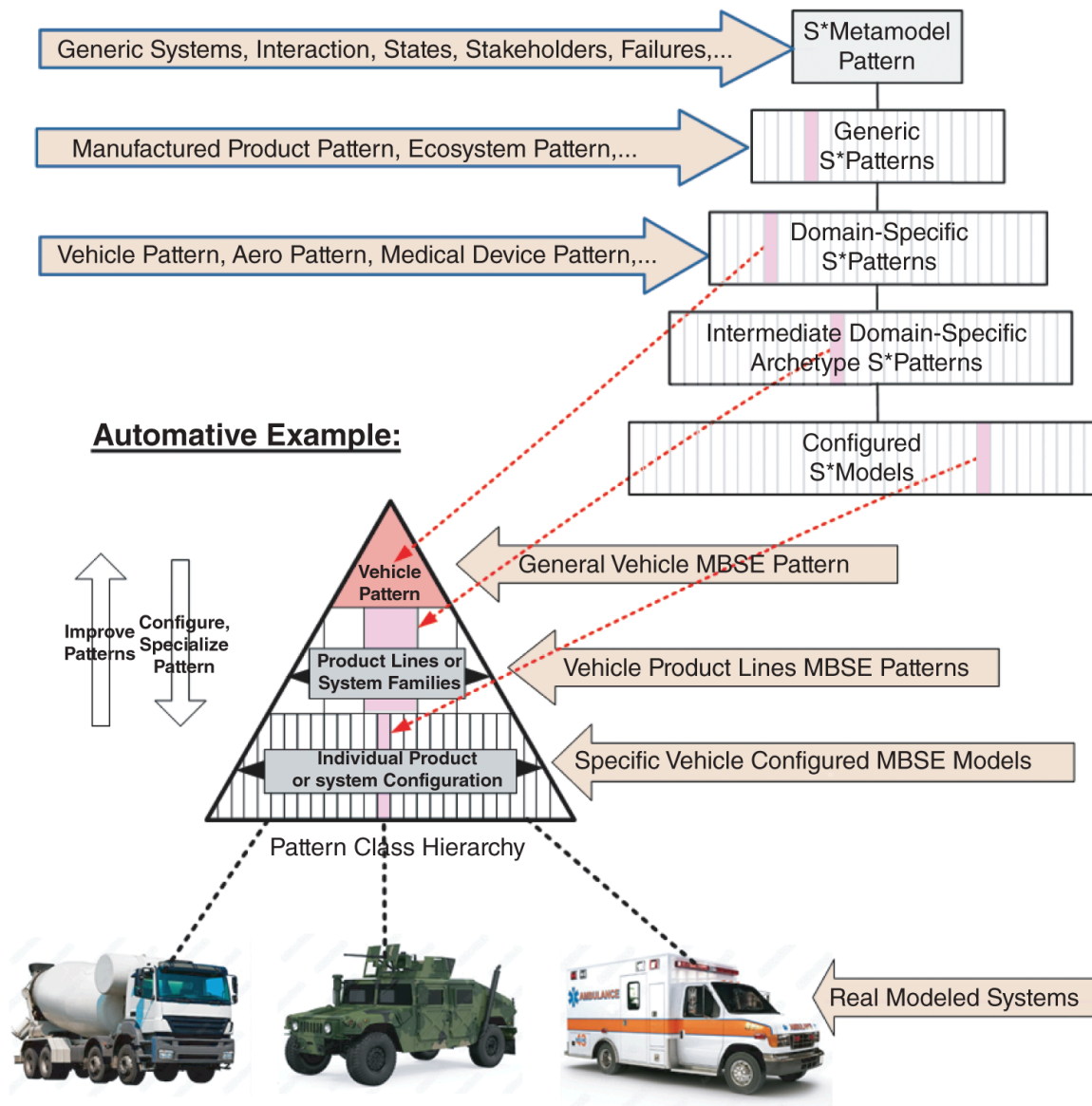
Other aspects of the S\*Metamodel, and the S\*Models generated from it, are described in Schindel (2011), INCOSE S\*MBSE (2022), and INCOSE S\*Patterns Primer (2022).

### ***S\*Patterns***

S\*Patterns are reusable S\*Models of families of systems, often domain-specific, configurable to represent multiple individual applications, market segments, or other configurations (Schindel, 2022a). There are also more generic S\*Patterns, such as the S\*Metamodel itself ( INCOSE S\*MBSE, 2022), the System Innovation Ecosystem Pattern introduced in [Figure 1.6](#) (Schindel and Dove, 2016) (INCOSE Innovation Ecosystems, 2022), and the



Model Characterization Pattern used to generate requirements and metadata for unified characterization of virtual models of all types (INCOSE S\*MCP, 2019). Pattern-based MBSE using S\*Patterns involves authoring of system patterns and their configuration to application and project-specific S\*Model instances, as summarized by [Figure 3.17](#). Part of the “minimality” of the S\*Metamodel is its sufficiency for such representations, including configuration rules. Instructional examples of system pattern representations may be found in Schindel and Peterson (2013). System patterns have been used in automotive, heavy equipment, aerospace, medical device, diesel and gas turbine engines, advanced manufacturing, consumer product, cybersecurity, and other domains (Bradley, et al., 2010) (Cook and Schindel, 2015) (Schindel and Smith, 2002) (Schindel, 2012).



**FIGURE 3.17** S\*Pattern class hierarchy. From (Schindel and Peterson, 2013). Used with permission. All other rights reserved.

There is more to pattern-based methods than just representing the patterns. Historical descriptions of SE processes can appear to describe all the processes practitioners ought to perform in order to discover, validate, and utilize all the information the system life cycle requires. However, those descriptions have by volume had less to say on the question of “what about what we *already* know?” Such descriptions might be viewed as relying on practitioners to separately work out informal means of exploiting existing knowledge within what the process specifies. To address such questions, the System Innovation Ecosystem Pattern shown in [Figure 1.6](#) describes the curation and mixing of information believed already credible with required new information extraction and validation. Schindel

and Dove (2016), INCOSE S\*Patterns Primer (2022) and others describe multiple additional levels of detail decomposition of processes, information, ecosystem capabilities, and limitations. Those details show how life cycle processes of ISO/IEC/IEEE 15288 (2023) and this handbook are incorporated to manage group learning and controlled sharing, and especially pattern credibility and uncertainty, across multiple programs of an enterprise, supply chain, or industry group. The System Innovation Ecosystem Pattern is further concerned with the effective linkage between the processes of pattern learning, validation, and curation versus the execution processes of making use of the content of trusted patterns—often by different people, at different times, in different places or organizations.

### **3.2.7 Design Thinking**

Understanding and leveraging the technical, business, and social relationships to successfully design and manage engineered systems is still a challenge in SE practice. SE solutions to this challenge tend to focus on system components, human activities, machine functionality, and human-system integration. Solution design can take advantage of Design Thinking (Dorst, 2015) as a complementary approach to Systems Thinking (see [Section 1.5](#)). Design Thinking explores (1) the human needs, (2) the operational and business processes and reasoning by which design concepts are devised and realized, especially those which are creative in nature, together with (3) the systems being realized, (4) its specialization, and (5) their utilities and value provided for the stakeholders.

In a Design Thinking process (Cross, 2000), (Lawson, 1997), context analysis and problem framing techniques are employed to identify all relevant influences on a problem, explore the given problem, and restructure or revise it to suggest a route to a solution. Solution generation techniques, including approaches to idea generation (ideation), are employed to identify a range of possible design solutions which are based on:

- existing known solutions, possibly in the form of variants, patterns or other adaptations;

- applying different forms of design-related reasoning to achieve innovative solutions;
- iterating between decomposing functional requirements and design solutions to achieve optimal design – see, for example, Axiomatic Design (Suh, 2001); and,
- using successive divergent and convergent phases of design synthesis and analysis with respect to the value provided for the stakeholders resulting from business and operational processes.

Design Thinking enables SE practitioners and other team members to understand the stakeholders, challenge assumptions, redefine problems, and realize innovative solutions by drawing upon logic, imagination, intuition, and systemic reasoning. Design Thinking can also be utilized for anticipating and addressing emergent features of systems, and in technical management and organization of engineering processes.

As Design Thinking approaches use solution-based methods, they can be used in various system life cycle stages. Examples are to support business or mission analysis (see [Section 2.3.5.1](#)), to identify and validate stakeholder or system requirements (see [Sections 2.3.5.2](#) and [2.3.5.3](#)), or to define the system architecture or its design solution (see [Sections 2.3.5.4](#) and [2.3.5.5](#)).

### **3.2.8 Biomimicry**

#### ***Definition***

*Natural systems* include living and non-living systems—anything that is not human-made. Natural systems differ from engineered systems (see [Section 1.1](#)), which are the primary focus of this handbook.

“*Biomimicry* is a practice that learns from and mimics the strategies found in nature to solve human design challenges—and find hope” (Biomimicry Institute, 2022).

## ***Purpose***

*Nature inspired SE* and biomimicry can improve processes, practices, and products through the understanding of how nature is structured, behaves, adapts, interacts, accomplishes functions, and recovers from disturbance. Applying natural systems thinking and engineering can improve system capability, efficiency, and performance, while benefiting operations, support, and the effects on external environments. Examples include optimized information processing and sensing, operation in extreme environments, innovative materials application, distributed architectures, understanding of how emergence arises, lowered environmental impact, and system resilience. Nature has strategies to improve performance in all these areas, including circular approaches to materials and energy. To utilize nature-inspired solutions, SE looks to a universal solution space and asks regularly, “Can nature help me solve this problem?” and, “How can nature help me improve my SoI, product, or process?”

## ***Examples***

Examples of successful natural systems applications and biomimicry abound. Select examples are shown in [Figure 3.18](#): Velcro® inspired by burdock (Velcro, 2023); an impeller inspired by the calla lily and nautilus shell (Pax Water Technologies, 2022); grippers inspired by the gecko (NASA JPL, 2013, 2014, 2015); and a sensor inspired by an insect’s compound eye (Frost, et al, 2016).



**FIGURE 3.18** Examples of natural systems applications and biomimicry. INCOSE SEH original figure created by McNamara and Anway derived from Studor (2016) and Hoeller, et al. (2016) using NASA images. Usage per the INCOSE Notices page. All other rights reserved.

### *Description*

Over time, natural systems have developed a very close fit to their surroundings and other systems. The result is that they exhibit optimized attributes that often exceed the performance of engineered systems. In addition, they often have positive impacts on the environment. The study of natural systems includes forms, structures, materials, behaviors, processes, regenerative strategies, and interactions. Studying natural systems will increase an SE practitioners' repertoire of solutions, architectural variations, and strategies. The SE practitioner on a project is ideally suited to explore opportunities for application of natural systems across all life cycle stages.

To develop natural systems solutions, the SE practitioner uses a systematic process that:

- Begins by being open to alternate solutions;
- Defines requirements in terms of abstract functions or goals, including specific relevant metrics whenever practical;
- In the early stages of solution exploration, uses the abstracted functions to search for and identify multiple natural systems that could satisfy the desired function and examines characteristics of each;
- Selects one or more candidate natural systems;
- Abstracts the strategy that accomplishes the function in nature;
- Explores architectural variations that translates the strategy and generates alternate system element alternatives;
- Transfers the strategy to the SoI;
- Evaluates system element performance at the system level; and
- Evaluates the environmental impact of the system production, operation, support, and retirement.

Partnering with and supporting natural systems scientists can be essential to a successful implementation. An SE team gains from the in-depth knowledge provided by a cross-disciplinary team.

For more information, see INCOSE NS Primer (2023).



# 4

# TAILORING AND APPLICATION CONSIDERATIONS

This section provides considerations for the application of SE with respect to different methodologies, approaches, system types, product sectors, and application domains.

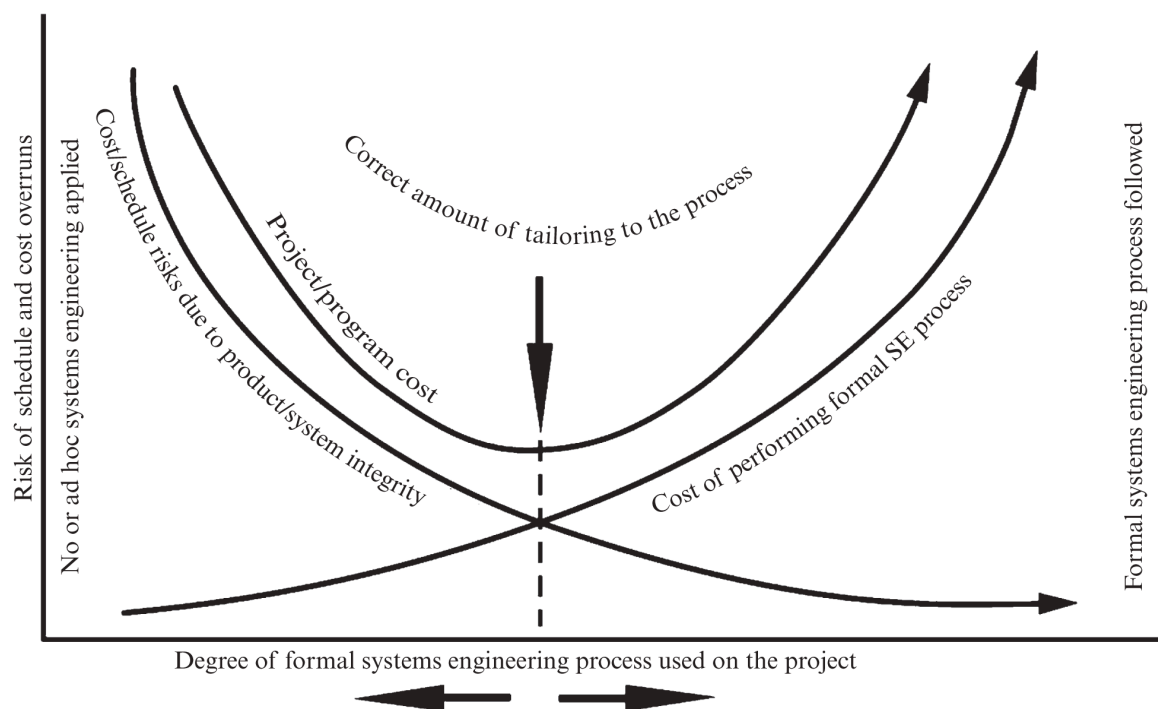
## 4.1 TAILORING CONSIDERATIONS

There are many standards and handbooks that address life cycle models and SE processes. However, in most cases, these cannot be directly applied to a given organization or project. There is usually a need to tailor them for the specific project, organization, environment, or other situational factors.

The principle behind tailoring is to adapt the processes to ensure that they meet the needs of an organization or a project while being scaled to the level of rigor that allows the system life cycle activities to be performed with an acceptable level of risk. In general, all system life cycle processes can be applied during all stages of the system life cycle, tailoring determines the process level that applies to each stage. Additionally, processes are applied iteratively, recursively, and concurrently as shown in [Figure 2.10](#).



Tailoring scales the rigor of application to an appropriate level based on risk. [Figure 4.1](#) is a notional graph for balancing formal process against the risk of cost and schedule overruns (Salter, 2003). Insufficient SE effort is generally accompanied by high risk of schedule and cost overruns. If too little rigor is applied, the risk of technical issues increases. However, as illustrated in [Figure 4.1](#), too much formal process may also lead to increased cost. If too much rigor is applied or unnecessary process activities or tasks are performed, the risk of cost and schedule slips increases. Tailoring occurs dynamically over the system life cycle depending on risk and the situational environment. Therefore, it should be continually monitored and adjusted as needed.



**FIGURE 4.1** Tailoring requires balance between risk and process. IncoSE SEH original figure created by Krueger adapted from Salter (2003). Usage per the IncoSE notices page. All other rights reserved.

This section describes the process of tailoring the system life cycle models and SE processes to meet organization and project needs.

## Tailoring Process

### Overview

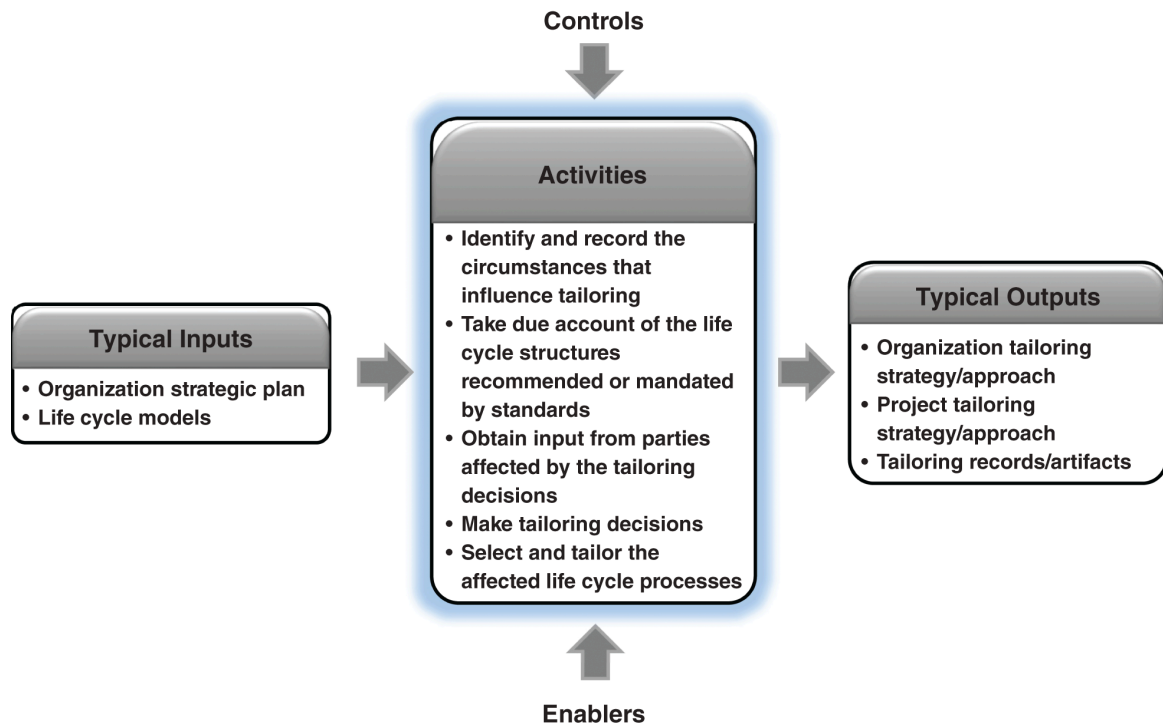
**Purpose** As stated in ISO/IEC/IEEE 15288,

[A.2.1] The purpose of the Tailoring process is to adapt the processes of ISO/IEC/IEEE 15288 (and of this handbook) to satisfy particular circumstances or factors that:

1. Surround an organization that is employing ISO/IEC/IEEE 15288 in an agreement;
2. Influence a project that is required to meet an agreement in which ISO/IEC/IEEE 15288 is referenced;
3. Reflect the needs of an organization in order to supply products or services.

**Description** At the organization level, the tailoring process adapts external standards in the context of the organizational processes to meet the needs of the organization. At the project level, the tailoring process adapts organizational processes for the unique needs of the project.

**Inputs/Outputs** Inputs and outputs for the Tailoring process are listed in [Figure 4.2](#). Descriptions of each input and output are provided in Appendix E.



**FIGURE 4.2** IPO diagram for Tailoring process. INCOSE SEH original figure created by Shortell, Walden, and Yip. Usage per the INCOSE Notices page. All other rights reserved.

**Process Activities** The Tailoring process includes the following activities:

- *Identify and record the circumstances that influence tailoring.*
  - – Identify the strategic, programmatic, and technical risks for the organization or project.
  - – Identify the level of novel concepts or the complexity of the system solution.
  - – Identify administrative effects (e.g., geographic distribution, organizational distribution, team size) that may impact tailoring.
  - – Record the relative importance of circumstances that influence tailoring.
  - – Identify tailoring criteria for each stage.
- *Take due account of the life cycle structures recommended or mandated by standards.*
  - – Identify relevant standards.
  - – Evaluate impact on tailoring and implementation across the life cycle.
- *Obtain input from parties affected by the tailoring decisions.*

- – Determine the level of expertise and experience of project members for the processes.
- – Determine the expectations or requirements from stakeholders.
- – Determine the risk tolerance of the stakeholders.
- *Make tailoring decisions.*
  - – Assess candidate life cycle models.
  - – Record assessment results.
- *Select and tailor the affected life cycle processes.*
  - – Capture and maintain rationale for selected life cycle processes.
  - – Establish means to continuously evaluate performance of tailored processes.

*Note: Tailoring can include the deletion, modification, or addition of outcomes, activities, tasks, typical inputs, or typical outputs.*

*Common approaches and tips:*

- Base decisions on facts and obtain approval from an independent authority.
- Use the Decision Management process to assist in tailoring decisions.
- Constrain the tailoring based on agreements between organizations.
- Control the extent of tailoring based on issues of compliance to stakeholder, customer, and organization policies, objectives, and legal requirements.
- Have different organizations propose and approve the tailoring.
- Influence the extent of tailoring of the agreement process activities based on the methods of procurement or intellectual property.
- Remove extra activities as the level of trust builds between parties.
- Identify the assumptions and criteria for tailoring throughout the life cycle to optimize the use of formal processes.
- Document the rationale for tailoring decisions.

## *Elaboration*

*Organizational Tailoring* Organizational tailoring applies specifically to the creating and maintaining organizational-level processes used by all projects. It is done in conjunction with the Life Cycle Model Management process (see [Section 2.3.3.1](#)). When contemplating if and how to incorporate a new or updated external standard into an organization, the following should be considered (Walden, 2007):

- Understand the organization;
- Understand the new standard;
- Adapt the standard to the organization (not vice versa);
- Institutionalize standards compliance at the “right” level;
- Allow for tailoring.

*Project Tailoring* Project tailoring applies specifically to the work executed through projects. It is done in conjunction with the Project Planning process (see [Section 2.3.4.1](#)). Factors that influence tailoring at the project level typically include, but are not limited to:

- Stakeholders and acquirers (e.g., number of stakeholders, quality of working relationships);
- Project budget, schedule, and requirements;
- Risk tolerance;
- Complexity and precedence of the system;
- The need for horizontal and vertical integration (see [Section 2.3.5.8](#)).

As mentioned in SE principle #12 (see [Section 1.4.3](#)), complex systems are engineered by complex organizations. Consequently, today’s systems are more often jointly developed by many different organizations. Cooperation must transcend the boundaries of any one organization. Harmony between multiple organizations is often best maintained by agreeing to follow a set of consistent processes, methods, and tools.

*Traps in Tailoring* Common traps in the tailoring process include, but are not limited to, the following:

- Reuse of a tailored baseline from another system without repeating the tailoring process;
- Using all processes and activities “just to be safe”;
- Assuming there is a single set of measures, risks, or other controls that apply to all projects without tailoring;
- Using a pre-established tailored baseline;
- Failure to include relevant stakeholders.

*Tailoring for Very Small Enterprises* The ISO/IEC/IEEE 29110 series defines Very Small Enterprises (VSEs) as enterprises, organizations, departments, or projects with up to 25 people. In many cases, VSEs find it difficult to apply international standards to their business needs and to justify the application of standards to their business practices. Typical VSEs do not have a comprehensive infrastructure, and the limited personnel usually are performing multiple roles. This may also happen in a large organization when the task is to perform a small project with less than 25 people involved. In this case, it can be extremely challenging to downscale the organization’s life cycle model that is designed for much larger projects.

The ISO/IEC/IEEE 29110 series defines guides for VSEs based on a set of VSE characteristics (e.g., business models, situational factors, risk levels). From that, four profiles were derived:

- *Entry* (less than 6 people or start-ups);
- *Basic* (single application by a single work team);
- *Intermediate* (more than one project in parallel with more than one work team); and
- *Advanced* (for VSEs that want to sustain and grow as an independent competitive system developer).

These profiles cover the needs of most VSEs. Each of the profiles defines subsets of international standards (e.g., ISO/IEC/IEEE 15288) relevant to the VSE’s respective context. For critical projects, such as mission critical or safety critical, these profiles do not apply, since the criticality of the projects would dictate a much greater level of rigor and comprehensive SE.

## 4.2 SE METHODOLOGY/APPROACH CONSIDERATIONS

The system definition activities, especially the partitioning of the system, requires that integration is considered throughout the development stage of the life cycle. The integration considerations may also require refinement based on when and how the work is performed. The choice of SE methodology or approach, along with the chosen life cycle model, often affects the sequence of work, which can help determine the focus of resources to address the unique challenges of the project.

This section introduces considerations for the following SE methodologies and approaches:

- Model-Based Systems Engineering (MBSE);
- Agile Systems Engineering;
- Lean Systems Engineering;
- Product Line Engineering (PLE).

Note that other types of SE methodologies and approaches exist.

### 4.2.1 Model-Based SE

This section provides an overview of the Model-Based Systems Engineering (MBSE) approach and includes a summary of its benefits relative to a more document-based approach. It also references a set of MBSE methodologies and provides a brief description of one representative methodology called the Object-Oriented Systems Engineering Method (OOSEM).

#### *MBSE Overview*

The *INCOSE Systems Engineering Vision 2020* (2007) defines MBSE as:

*The formalized application of modeling to support system requirements, design, analysis, verification, and validation activities beginning in the [concept stage] and continuing throughout development and later life cycle [stages].*

MBSE is often contrasted with a document-based approach to SE. In a document-based SE approach, there is often considerable information generated about the system that is contained in documents and other artifacts such as specifications, interface control documents, system description documents, trade studies, analysis reports, and verification plans, procedures, and reports. The information contained within these documents is often difficult to synchronize and maintain, and difficult to assess in terms of its quality (correctness, completeness, and consistency). Although many systems have been developed using a traditional document-based approach, a model-based approach is becoming essential to address the increasing complexity of systems and support approaches that can more effectively and efficiently adapt to requirements and design changes.

MBSE enhances the ability to capture, analyze, share, and manage the information associated with the specification of a product that can result in the benefits listed below. There is some quantitative data (Rogers and Mitchell, 2021) and considerable qualitative data (OMG MBSE Wiki, 2023, MBSE Events and Related Meetings) from industry papers and presentations that support the following benefits of MBSE:

- *Improved communications* among the development stakeholders (e.g., the acquirer, project management, SE practitioners, hardware and software developers, testers, quality characteristic disciplines).
- *Increased ability to manage system complexity* by enabling the system to be viewed from multiple perspectives.
- *Improved product quality* by providing an unambiguous and precise model of the system that can be evaluated for consistency, correctness, and completeness.
- *Reduced cycle time* by enabling better control of the technical baseline, more rapid impact analysis, improved specification and design reuse,



early insight for design decisions, and early discovery of potential defects.

- *Reduced risk by surfacing requirements and design issues early.*
- *Enhanced knowledge capture and reuse of the information by capturing information in more standardized ways and reducing redundancy of information.*
- *Improved ability to teach and learn SE fundamentals by providing a clear and unambiguous representation of systems and system concepts.*

### ***MBSE Methodologies***

In an MBSE approach, much of the information that has been traditionally captured in informal diagrams, text, and tables is captured in a *descriptive system model* (see [Section 3.2.1](#)). This includes information about the system context, the requirements on the system and its elements, the system architecture including its structure and behavior, the critical parameters needed to specify the analysis of the system, and information about how the system is verified to satisfy its requirements. Modeling languages such as SysML™ (OMG SysML, 2021) are often used to capture this information in a standard way (see [Section 3.2.1](#)). The system descriptive model is augmented by other models, such as models to capture the system geometric configuration and various analytical models, to analyze the performance and other quality characteristics of the system. Each kind of model captures different kinds of information about the system. The different models must be managed as the design evolves to ensure a coherent representation of the overall system.

In an MBSE approach, the system descriptive model is a primary artifact of the SE process. MBSE formalizes the application of SE by creating the system descriptive model and integrating it with the other kinds of models. The kind of information and the level of detail of the information that is captured in models and maintained throughout the life cycle depends on the scope of the MBSE effort. An effective MBSE methodology supported by appropriate tools and a team with the requisite SE skills and knowledge are essential to fully realize the benefits of MBSE.

An *MBSE methodology* describes how MBSE is performed to capture the required information in the system descriptive model and related artifacts. Like any methodology, it must be tailored to the particular need of the organization and/or project (see [Section 4.1](#)). This includes defining the appropriate life cycle model, tailoring the activities and work products to align with the project scope and modeling objectives, and selecting the appropriate tools to create and manage the models and other relevant data. Estefan (2008) published a survey of candidate MBSE methodologies under the auspices of an INCOSE technical publication. Information on these methodologies is available on the Methodology and Metrics web page of the INCOSE MBSE Wiki (2022). These methodologies and others continue to evolve based on their application to real world projects. OOSEM is summarized below as a representative MBSE method.

### ***OOSEM Summary***

OOSEM is an MBSE method intended to help architect systems that satisfy evolving mission and system requirements and can accommodate changes in technology and design. OOSEM is generally consistent with processes in this handbook. It can be adapted to different life cycle models to support the specification, analysis, design, and verification of systems. The method enables the flow-down of requirements from mission, to system, to system element levels, which are realized by applicable hardware, software, data, and other discipline-specific design methods.

OOSEM describes fundamental SE activities whose outputs are model-based artifacts. The modeling artifacts are captured in a system descriptive model using the Systems Modeling Language (SysML™) along with other analytical models. A process model for OOSEM can be downloaded from the INCOSE OOSEM Working Group website (2022).

The OOSEM supports a development process that includes the following subprocesses and activities:

- *Manage the system development*—activities include plan and control the technical effort, including planning, risk management, configuration management, and other project monitoring and control activities;
- *Specify and design the system*—activities include analyze stakeholder needs, specify the system requirements, develop the system architecture, and allocate the system requirements to system elements;
- *Develop the system elements*—activities include design the elements, implement the elements, and verify the elements satisfy the allocated requirements; and
- *Integrate and verify the system*—activities include integrate the system elements and verify that the integrated system elements satisfy the system requirements.

This OOSEM process can be applied at each level of the system hierarchy to specify the requirements of the system and its elements. Applying the process recursively at successive levels of the hierarchy may involve multiple iterations throughout the development process. To be effective, the fundamental tenets of SE must be applied including the use of multi-disciplinary teams and a disciplined management process.

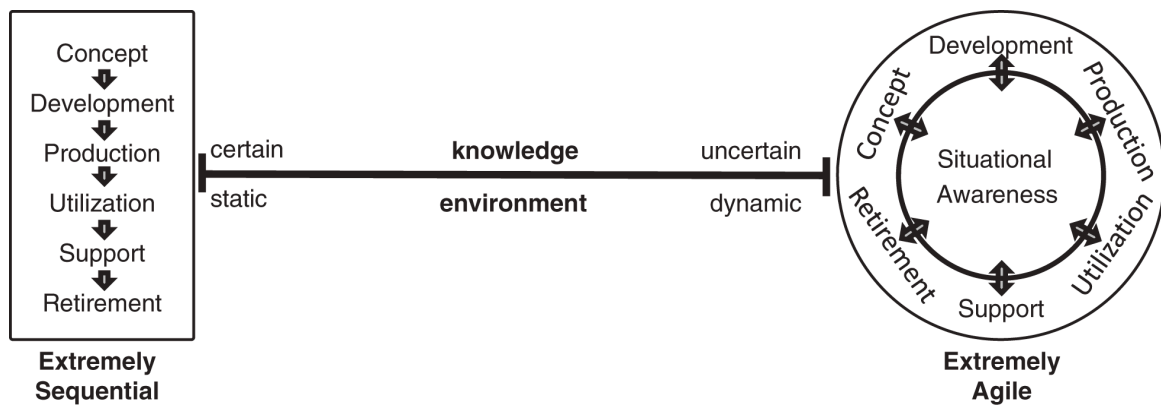
#### **4.2.2 Agile Systems Engineering**

The knowledge of requirements for an effective system often continues to change during the system life cycle. Common causes include insufficient initial knowledge, new knowledge revealed during development and utilization, and continual evolution in the targeted operational environment of the system. When evolution of the system's operational environment doesn't stop with initial deployment, a system's functional capabilities must evolve if it is to remain viable. Under these circumstances system engineering is virtually never ending, and retirement is generally an issue of safe and secure functional capability disposal rather than system decommissioning.

*Agile Systems Engineering* is a principle-based approach for designing, building, sustaining, and evolving systems when knowledge is uncertain, or

environments are dynamic. Thus, Agile System Engineering is a what, not a how. As stated in [Section 2.2](#), there are many life cycle models (e.g., Vee, Incremental Commitment Spiral Model (ICSM), DevSecOps (Development, Security, Operations)). Some of them are targeted on a single engineering domain (e.g., XP (Extreme Programming), Scrum, DevOps (Development, Operations), and various scaled approaches such as SAFe (Scaled Agile Framework) in the software engineering domain). Most of them have a strong focus on the development stage.

Agile Systems Engineering is best understood when contrasted to the sequential life cycle approach and in how the two relate to the system life cycle spectrum. [Figure 4.3](#) shows extreme forms of these two life cycle approaches in terms of their activity stages and data flows. All life cycle approaches fall somewhere between the two ends of the spectrum, depending upon the process-encoded degree of attentiveness and responsiveness to dynamics in knowledge and environment. It is unlikely that either depicted extreme would be effective in actual practice.



**FIGURE 4.3** SE life cycle spectrum. From dove (2022). Used with permission. All other rights reserved.

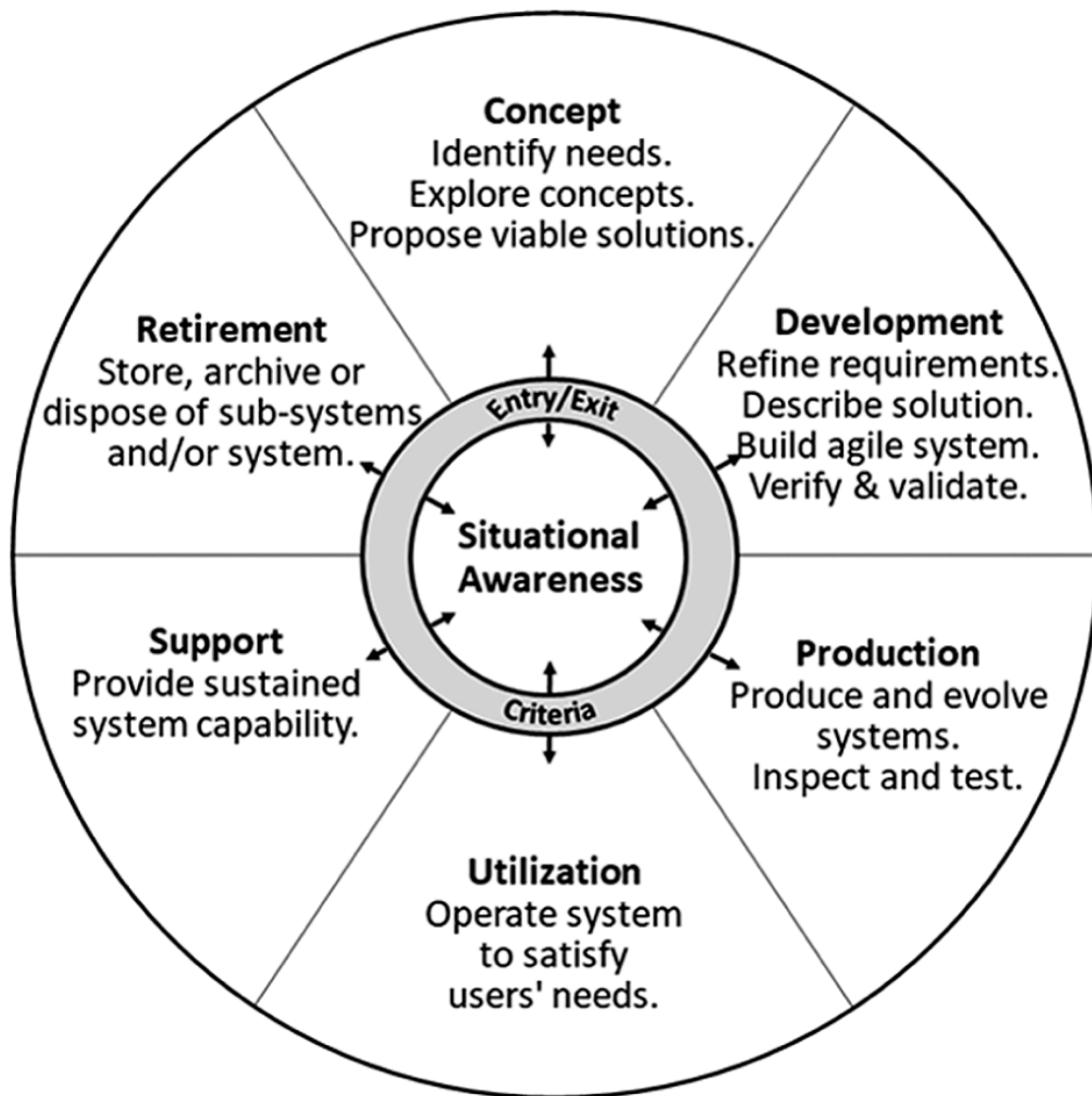
An Agile Systems Engineering process is based on strategies for timely and continual knowledge development and affordable application of new knowledge in system development activity. Virtually all forms of Agile Systems Engineering employ incremental or evolutionary development in some way (see [Sections 2.2.2](#) and [2.2.3](#)) as a means to produce demonstrable and/or usable work in process that provokes feedback for real time learning and subsequent application.

Software plays an increasingly major role in most systems today. Codified agile software development methodologies offer relevant approaches for rapid knowledge discovery and deployment in the software domain.

Patterns from these software approaches can inform agile engineering approaches in other domains and in the encompassing domain of SE; but each domain has unique differences (e.g., external dependencies, fabrication techniques, development cycle time constraints, development support tools).

### *Agile Systems Engineering Life Cycle Model*

The Agile Systems Engineering life cycle model is depicted on the far right of [Figure 4.3](#) and with more detail in [Figure 4.4](#). The six system life cycle stages run around the perimeter, with situational awareness featured in the center. The agile life cycle model can accommodate activities in any and all of the stages concurrently without progressive sequencing. [Figure 4.4](#) depicts a life cycle model (Dove and Schindel, 2019), not to be confused with the Vee model (see [Section 2.2.1](#)) which depicts relationships among SE activities applicable in sequential, iterative, and evolutionary approaches.



**FIGURE 4.4** Agile SE life cycle model. From dove and Schindel (2019). Used with permission. All other rights reserved.

Situational awareness has no entry or exit criteria, as it should, in principle, be a continuous activity. Entry criteria for all of the other stages begins with a decision to act upon specific triggering awareness, and may require process-prudent, or contract-required, engagement criteria for a stage or stages to be entered. An Agile Systems Engineering process is predicated upon real-time experimentation and learning in all stages, and as such, the entry criteria may be as simple as the decision to enter a stage for experimental knowledge development that may or may not produce artifacts for use in other stages. On the other hand, exit criteria for a stage that produces artifacts for use in other stages should have some fixed requirements for satisfactory stage completion, with recognition that the

outcome of stage activity may simply be valuable learned knowledge that aborts the need for producing artifacts of use in other stages.

The retirement stage deals with system elements and older system versions that are retired frequently, as the “current” system evolves. This has implications for safe and secure maintenance, disposal, and reversion processes.

Fleshing out a generic Agile SE Life Cycle Model for a specific project likely starts with default standard processes in each stage, tailored and augmented for specific agile SE differences. Adapting the generic model to a specific organization’s process will tailor and augment the generic model as the organization’s standard process evolves.

### ***Agile Systems Engineering Life Cycle Operational Considerations***

As described in [Section 1.3.4](#), [Figure 1.6](#) depicts the life cycle operational “pattern” as three nested systems:

- **System 1 – The Engineered System** is the target system under development.
- **System 2 – The Life Cycle Project Management System** includes the basic SE development and maintenance processes, and their operational domains that produces System-1.
- **System 3 – The Enterprise Process and Innovation System** is the process improvement system that learns, configures, and matures System-2. System 3 is responsible for situational awareness, evolution, and knowledge management, the provider of operational agility. Intent is continuous, not episodic, information flow among the three systems. Principles and strategies that facilitate operational agility in action include:

#### ***Sensing:***

- External awareness (proactive alertness);
- Internal awareness (proactive alertness);

- Sense making (risk analysis, trade space analysis).

*Responding:*

- Decision making (timely, informed);
- Action making (invoke/configure process activity to address the situation);
- Action evaluation (verification and validation).

*Evolving:*

- Experimentation (variations on process ConOps);
- Evaluation (internal and external judgement);
- Memory (evolving culture, response capabilities, and process ConOps).

The architecture and structural principles that enable system agility are covered in [Section 3.1.3](#).

### *Agile Systems Engineering Examples*

Agile system engineering methodologies are project-context dependent. What is common across methodologies are certain fundamental strategies that get tailored for a specific context. Four published examples illuminate this tailoring in four different contexts:

- As shown in [Figure 2.9](#), Rockwell Collins employed a product line engineering (PLE) (see [Section 4.2.4](#)) approach for a large family of radio products composed of software, firmware, and electronic circuit boards with a continuous integration platform that accommodated asynchronous evolution of mixed-domain system element work in process (Dove, et al., 2017).
- The US Navy SpaWar delivered innovative off-road autonomous vehicle technology in continuous six-month development increments with parallel tracks of integration, test, and architecture evolution (Dove, et al., 2016).



- Northrop Grumman evolved user capabilities in six month increments for a software systems-of-systems single-point hub that provided access to 22 independent logistics data bases, with three successive generations under active life cycle control at all times (Dove and Schindel, 2017).
- Lockheed Martin evolved F16, F22, and F35 weapons capabilities with internally developed software and externally subcontracted hardware in roughly six month development increments in a tailored SAFe approach; featuring a continuous integration and demonstration platform with asynchronous evolution of system element simulations, low fidelity proxies, work in process, and completed system elements (Dove, et al., 2018).

### 4.2.3 Lean Systems Engineering

SE is regarded as an established, sound practice, but is not always delivered efficiently. US Government Accountability Office (GAO, 2008) and NASA (2007a) studies of space systems document major budget and schedule overruns. Similarly, studies by the MIT-based Lean Advancement Initiative (LAI) have identified a significant amount of waste in government projects, averaging 88% of charged time (LAI MIT, 2013; McManus, 2005; Oppenheim, 2004; Slack, 1998). Most projects are burdened with some form of *waste*: politicization, poor coordination, premature and unstable requirements, quality problems, and management frustration. This waste represents a vast productivity reserve in projects and major opportunities to improve project efficiency.

*Lean system development* and the broader methodology of *lean thinking* have their roots in the Toyota “just-in-time” philosophy, which aims at “producing quality products efficiently through the complete elimination of waste, inconsistencies, and unreasonable requirements on the production line” (Toyota, 2009). *Lean SE* is the application of lean thinking to SE and related aspects of organization and project management. SE is the discipline that enables flawless development and integration of complex technical systems. Lean thinking is a holistic paradigm that focuses on delivering maximum value to the customer and minimizing waste. A

popular description of lean is “doing the right job right the first time.” Lean thinking has been successfully applied in manufacturing, healthcare, administration, supply chain management, and product development, including engineering.

Lean SE is the area of synergy between lean thinking and SE, with the goal to deliver the best life-cycle value for technically complex systems with minimal waste. The early use of the term lean SE is sometimes met with concern that this might be a “repackaged faster, better, cheaper” initiative, leading to cuts in SE at a time when the profession is struggling to increase the level and quality of SE effort in projects. Lean SE does not take away anything from SE and it does not mean *less* SE. It means *better* SE with higher responsibility, authority, and accountability, leading to better, waste-free workflows with increased mission assurance.

Three concepts are fundamental to the understanding of lean: value, waste, and the process of creating value without waste (captured into lean principles).

### ***Value***

The value proposition in engineering projects is often a multiyear, complex, and expensive acquisition process involving numerous stakeholders and resulting in hundreds or even thousands of requirements, which, notoriously, are rarely stable. In lean SE, *value* is defined simply as mission assurance (i.e., the delivery of a flawless complex system, with flawless technical performance, during the product or mission development life cycle) and satisfying the customer and all other stakeholders. This implies completion with minimal waste, minimal cost, and the shortest possible schedule.

### ***Waste in Product Development***

*Waste* is “the work element that adds no value to the product or service in the eyes of the customer. Waste only adds cost and time” (Womack and Jones, 1996). The LAI classifies waste into seven categories (McManus,

2005). An eighth category, the waste of human potential, is increasingly added. These categories are defined and illustrated as follows.

- *Overprocessing*—Processing more than necessary to produce the desired output; excessive refinement, beyond what is needed for value.
- *Waiting*—Waiting for people, material or information, or people waiting for information or material; late delivery of material or information, or delivery too early—leading to eventual rework.
- *Unnecessary movement*—Moving people (or people moving) unnecessarily to access or process material or information; unnecessary motion in the conduct of the task; lack of direct access; manual intervention.
- *Overproduction*—Creating too much material or information; performing a task that nobody needs; information over-dissemination and pushing data.
- *Transportation*—Moving material or information unnecessarily; unnecessary hand-offs between people; incompatible communication—lost transportation through communication failures.
- *Inventory*—Maintaining more material or information than needed; too much “stuff” buildup; complicated retrieval of needed “stuff”; outdated, obsolete information.
- *Defects*—Errors or mistakes causing the effort to be redone to correct the problem.
- *Waste of human potential*—Not utilizing or even suppressing human enthusiasm, energy, creativity, and ability to solve problems and general willingness to perform excellent work.

### ***Lean Principles and Lean Enablers for Systems Engineering***

Womack and Jones (1996) captured the *process of creating value without waste* into six *lean principles* described in (Oppenheim, 2011), as follows:

*The value principle promotes a robust process of establishing the value of the system to the customer with crystal clarity early in the project. The process should be customer-centric, involving the customer frequently and aligning employees accordingly.*

*The value stream principle emphasizes detailed project planning and waste-preventing measures, solid preparation of the personnel and processes for subsequent efficient workflow, and healthy relationships between stakeholders (e.g., acquirer, contractor, suppliers, and employees); project frontloading; and use of leading indicators and quality measures. SE practitioners should prepare for and plan all end-to-end linked actions and processes necessary to realize streamlined value, after eliminating waste.*

*The flow principle promotes the uninterrupted flow of robust quality work and first-time-right products and processes, broad steady competence instead of hero behavior in crises, excellent communication and coordination, concurrency, frequent clarification of the requirements, and making project progress visible to all.*

*The pull principle is a powerful guard against the waste of rework and overproduction. It promotes pulling tasks and outputs based on internal and external customer needs (and rejecting others as waste), and better coordination between the pairs of employees handling any transaction before their work begins so that the result can be first-time right.*

*The perfection principle promotes excellence in the SE and organization processes, utilization of the wealth of lessons learned from previous projects into the current project, the development of perfect collaboration policy across people and processes, and driving out waste through standardization and continuous improvement. Imperfections should be made visible in real time, and continuous improvement tools (Six Sigma) should be applied as soon as possible. It calls for a more important role of SE practitioners, with*

*responsibility, accountability, and authority for the overall technical success of the project.*

*Finally, the respect-for-people principle promotes the enterprise culture of trust, openness, honesty, respect, empowerment, cooperation, teamwork, synergy, and good communication and coordination and enables people for excellence.*

In 2011, a project undertaken jointly by PMI, INCOSE, and the LAI at MIT developed the Lean Enablers for Managing Engineering Programs (LEfMEP, 2012), adding lean enablers for project management and holistically integrating lean project management with lean SE. A major section of the book is devoted to a rigorous analysis of challenges in managing engineering projects. They are presented under the following 10 challenge themes:

1. Firefighting—reactive project execution;
2. Unstable, unclear, and incomplete requirements;
3. Insufficient alignment and coordination of the extended enterprise;
4. Processes that are locally optimized and not integrated for the entire enterprise;
5. Unclear roles, responsibilities, and accountability;
6. Mismanagement of project culture, team competency, and knowledge;
7. Insufficient project planning;
8. Improper metrics, metric systems, and key performance indicators;
9. Lack of proactive project risk management; and
10. Poor project acquisition and contracting practices.

#### **4.2.4 Product Line Engineering (PLE)**

Rarely does anyone build just one edition, just one flavor, just one point solution of anything. In many cases, SE is performed in the context of a product line—a family of similar systems with variations in features and functions. *Product Line Engineering* (PLE) addresses this mismatch by

providing models, tools, and methods for holistic engineering of system families.

*A note on terminology:* Where the PLE field and standards refer to “product,” “product line,” and “product line engineering,” the equivalent terms in SE are “system,” “system family,” and “system family engineering,” respectively. These terms can be used interchangeably (Krueger, 2022).

### ***Challenges with Early Generation System Family Engineering Approaches***

When systems in a *system family* are engineered as individual point solutions, techniques such as *clone-and-own reuse* or *branch-and-merge* result in ever-growing duplicate and divergent engineering effort. Trying to manage the commonality and variability among these individually engineered systems in the family has traditionally relied on tribal knowledge and high bandwidth, error-prone interpersonal communication. Furthermore, when each engineering discipline adopts a different ad hoc technique for managing variations among the members of the system family, the result is error prone dissonance when trying to translate and communicate across the different life cycle disciplines.

This is a self-inflicted complexity, over and above the complexity inherent in the systems being engineered. It consumes engineering teams with low-value, trivial, replicative work that deprives them of time and energy that would be better spent on high-value innovative work that advances system and business objectives.

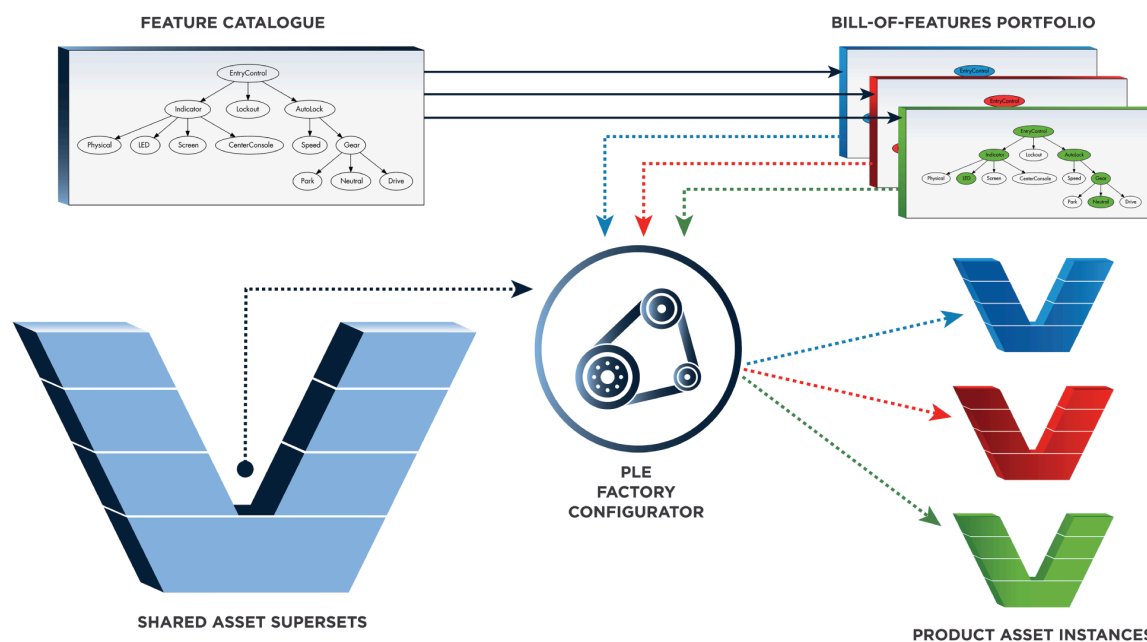
### ***Feature-based Product Line Engineering***

*Feature-based PLE* is the modern digital engineering industry good practice for PLE, as defined in the INCOSE PLE Primer (2019) and ISO/IEC 26580 (2021). Feature-based PLE offers significant improvements and benefits in effort, cost, time, scale, and quality by exploiting system similarity while formally managing variation.

Feature-based PLE is used to engineer a system family as a single holistic system rather than a multitude of individual systems. Engineering assets in each engineering discipline are consolidated to eliminate duplication and divergence. A single authoritative variation management model is applied consistently across all assets in all engineering disciplines to eliminate that source of dissonance across the life cycle and to enable organizations to make informed and deliberate cost-benefit decisions about the variations designed into their system family.

### *Key Elements of a Feature-based PLE Factory*

Feature-based PLE uses a *PLE Factory* metaphor, as illustrated in [Figure 4.5](#). See ISO/IEC 26580 (2021) for a full description.



**FIGURE 4.5** Feature-based pLE factory. From IncoSE pLE primer (2019). Usage per the IncoSE notices page. All other rights reserved.

- **Feature Catalogue**, as shown in the upper left, captures a formal model of the distinguishing characteristics about how the members of the system family differ from each other and provides a common language and single authoritative source of truth about variation throughout the engineering organization.
- **Bill-of-Features**, as shown in the upper right, specifies the features selected from the Feature Catalogue for each system in a system family

portfolio.

- **Shared Asset Supersets**, as shown in the lower left, are the engineering artifacts that support the creation, design, implementation, deployment, and operation of systems in a system family. They contain *variation points*, which are pieces of content that can be included, omitted, generated, or transformed for a system instance, based on the features selected in a Bill-of-Features for that system.
- **PLE Factory Configurator**, shown in the center, is an automation that applies a Bill-of-Features for a system to each variation point in the Shared Asset Supersets, to determine each variation point's content for the system instance.
- **Product Asset Instances**, shown in the lower right, each contain only the shared asset content suited for that one system in the system family.

With Feature-based PLE, engineers now work in the PLE Factory on the Shared Asset Supersets, the Feature Catalogue, and the Bills-of-Features rather than on the individual system instances. Once the PLE Factory is established, engineering assets for the individual systems are automatically instantiated rather than manually engineered. Feature-based PLE transforms the task of engineering a plethora of individual systems into the much more efficient task of producing a single system: The PLE Factory itself. This consolidation also means that change management and configuration management are performed on the single PLE Factory rather than separately on each of the system instances.

### ***Shared Asset Supersets and Variation Points***

To work in a PLE Factory, engineers must learn how to create and maintain Shared Asset Supersets, including variation points, for their discipline. For example, requirements engineers learn how to create requirements Shared Asset Supersets with variation points, test engineers learn how to create verification and validation Shared Asset Supersets with variation points, and software engineers learn how to create source code Shared Asset Supersets with variation points.



A Shared Asset Superset contains a single copy of all content used in any system—that is, there is no duplication of content. Content that appears in every system is said to be *common* content, while content that varies from system to system is encapsulated in a variation point. Consistent treatment of variation points in Shared Asset Supersets across all disciplines is a hallmark of Feature-based PLE. Variation points are places in an asset that denote content that is configured according to feature selections in a Bill-of-Features for a particular system instance. Variation point configuration mechanisms typically include selection or omission of the content; selection from among mutually exclusive content alternatives; generation of content based on feature specifications; and feature-based transformation of content from one form into another.

Perhaps one of the easiest examples of Shared Asset Supersets to understand is requirements. A superset of requirements combines individual system requirements to establish all of the system family requirements. Variation points express inclusion and omission, define mutual exclusion, and transform requirement wording in the system specification—all based on feature selections. Requirement transformation can replace numbers, units, or other text with information that is derived from the Bill-of-Features. Requirements that have no variation are common and apply to every system.

MBSE models can be developed as Shared Asset Supersets and instrumented with variation points. For example, system design or architecture models using SysML™ include variation points to specify optional, mutually exclusive, and varying structural elements such as blocks, ports, relationships, objects, classes, activities, states, transitions, use-cases, packages, and others, as well as generation or transformation of values, attributes, and constraints associated with those elements.

Shared Asset Supersets for Electronic Design Automation, Mechanical Design Automation, and Computer-aided Design (CAD) for electronic, mechanical, mechatronic, and cyber-physical systems take the form of supersets of parts, properties, relationships, assemblies, system elements,

circuit boards, wiring harnesses, and more. Variation points instrument optional, mutually exclusive, and varying content in these models.

In software systems, Shared Asset Supersets are constructed for source code, resources, and build scripts. Source code variation points can be defined in several ways, including blocks of code, optional or mutually exclusive source files, and macro substitutions.

Verification and validation Shared Asset Supersets for automated and manual test plans and test cases can be instrumented with variation points to identify and configure the tests for each system, based on feature selections. It is possible to streamline or even eliminate redundant testing of common capability across multiple systems in the system family.

A broad array of additional assets with digital representations can serve as Shared Asset Supersets in system families. These include system budgets or cost models, schedules and work plans, user manuals and installation guides, process documentation, marketing brochures, simulation models, system descriptions, digital twins, supply chain orders, manufacturing specs, contract proposals, and much more. Feature-based PLE can be applied to all SoI types defined in [Section 4.3](#).

### ***Organizational Change and Return-on-Investment with Feature-based Product Line Engineering***

For many organizations, Feature-based PLE represents a shift in engineering approach that requires organizational change, along with commitment from engineering and business leadership to make that change. The ROI to justify the organizational change is in most cases compelling, based on the elimination of low-value, mundane, replicative work, with doubling, tripling and larger improvements in engineering metrics such as: lowering engineering complexity; reducing overall engineering time, cost, and effort; increasing portfolio scalability; and improving system quality (Gregg, et al., 2015 and McNicholas, 2021). In consideration of this ROI, the question to leadership is, “What if your

engineers could do their normal day's work before lunch; what would you have them do in the afternoon?" There are many answers to this question, all of them good.

## 4.3 SYSTEM TYPES CONSIDERATIONS

The concept of SoI was introduced in [Section 1.3.1](#). The type of SoI has significant implications on SE. This section introduces SE considerations for the following types of SoIs:

- Greenfield/Clean Sheet Systems
- Brownfield/Legacy Systems
- Commercial-off-the-Shelf (COTS)-Based Systems
- Software-Intensive Systems
- Cyber-Physical Systems (CPS)
- System of Systems (SoS)
- Internet of Things (IoT)/Big Data-Driven Systems
- Service Systems
- Enterprise Systems

Note that other types of SoIs exist.

### 4.3.1 Greenfield/Clean Sheet Systems

“Greenfield” and “brownfield” are terms used in real estate. Greenfield land is previously undeveloped space, such as a (green) farmer's field. Brownfield land has been previously developed, typically has existing structures and services in place, and may contain undesirable or hazardous materials (also known as waste) that must be remediated. Greenfield SE, also known as “clean sheet” or “blank slate” SE, involves systems that are new designs and have no, or limited, legacy systems constraints, other than system interfaces. Given the incremental and spiral development life cycle approaches of today, a greenfield system may evolve toward brownfield even before it is delivered, from the developer's perspective.

Traditionally, SE has been taught by considering systems from a greenfield perspective. One starts with a “clean or blank sheet of paper” and determines the set of stakeholders and their needs and requirements, translates them into system requirements, architects and designs a system solution, implements the system elements, and then integrates, verifies, and validates the system elements and the system solution. While this is an effective way to teach SE and to prepare practitioners with skills applicable to the entire system life cycle, few system development efforts are truly greenfield. Greenfield, therefore, is an almost theoretical situation that is rarely seen in practice. The remaining considerations provide different perspectives and implications beyond greenfield SE.

Sometimes, it can be quite traumatic for organizations that make brownfield updates to its legacy products over a long period of time to transition from brownfield development back to greenfield (Axehill, 2021). They may need to “relearn” how to do greenfield.

#### **4.3.2 Brownfield/Legacy Systems**

As described in [Section 4.3.1](#), brownfield (and greenfield) are terms used in real estate. Brownfield land has been previously developed, typically has existing structures and services in place, and may contain undesirable or hazardous materials (also known as waste) that must be remediated.

Brownfield SE, also known as “legacy” SE, involves significant modifications, extensions, or replacement of an existing “as-is” system in an existing environment to an updated “to-be” system. Brownfield systems often contain waste (e.g., technical debt) that may need to be remediated (Seacord, et al., 2003) (Hopkins and Jenkins, 2008). “In-service” systems are another example of brownfield system (Kemp, 2010) (Van De Ven, et al., 2012). Brownfield systems typically have explicit continuity requirements, where the operation of the as-is system needs to continue, resulting in a deliberate transition to the updated system.

The nature of greenfield and brownfield systems drives different life cycle approaches that reflect different areas of emphasis. [Table 4.1](#) lays out some

of the key differences across a set of aspects important to SE (Walden, 2019) (Baley and Belcham, 2010). This impacts not only the system solution, but also the team that is put in place to develop the system. As with all development efforts, SE processes need to be tailored to fit the needs of a given project (see [Section 4.1](#)). SE in a brownfield environment augments the SE life cycle processes described in this handbook with site surveys and reconstruction activities to understand the as-is systems, identify gaps, and engineer the to-be system (Walden, 2019).

**TABLE 4.1** Considerations of greenfield and brownfield development efforts

| Aspect                               | Greenfield                                    | Brownfield                                                                                                                      |
|--------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Life Cycle Stage(s) (of Initial SoI) | Concept, Development                          | Utilization, Support                                                                                                            |
| Focus                                | New or novel features                         | Maintenance or adding new features while retaining select legacy functionality                                                  |
| Maturity (of Initial SoI)            | Low to Moderate                               | High for maintenance; Mix for existing system and environment, plus new development for upgrade or replacement                  |
| Architecture and Design Review       | Reviewed and modified at multiple levels      | Reviewed only when significant updates are made/performed                                                                       |
| Verification                         | The entire SoI typically needs to be verified | Only the updated and impacted parts of the system need to be verified (there may be regression testing for the unchanged parts) |

| Aspect                    | Greenfield                                                                                          | Brownfield                                                                                                                                                             |
|---------------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Validation                | The entire SoI typically needs to validated with the customer/user                                  | The entire SoI (including changes) typically needs to validated with the customer/user to check for new emergent behavior                                              |
| Manufacturing/Production  | May be in place if using the existing line, or is developed (or tailored) as development progresses | Mostly in place, reverse engineering of existing designs may be required if the original design can no longer be produced (e.g., due to as-is use of banned materials) |
| Maintenance and Logistics | Developed (or tailored) as development progresses                                                   | Mostly in place, but may need changes or upgrades depending on the replacement system elements                                                                         |
| Practices and Processes   | Developed (or tailored) as work progresses                                                          | Mostly in place, though not necessarily relevant to the new team                                                                                                       |
| Team Composition          | Newly formed group                                                                                  | Mix of old and new, bringing both                                                                                                                                      |

| Aspect | Greenfield | Brownfield                        |
|--------|------------|-----------------------------------|
|        |            | historical biases and fresh ideas |

From Walden (2019) derived from Baley and Belcham (2010). Used with permission. All other rights reserved.

### 4.3.3 Commercial-off-the-Shelf (COTS)-Based Systems

One of the key trade-off studies SE practitioners perform is the “make vs. buy” decision on system elements. “Make” represents custom-built solutions; “buy” represents outsourced development and commercial-off-the-shelf (COTS) solutions. Directed use also can result in COTS. Most systems have some COTS content. The following characteristics can be useful when deciding if a particular system or system element can be characterized as COTS (Oberndorf, et al., 2000) (Tyson, et al., 2003):

- Sold, leased, or licensed to the general public;
- Offered by a vendor trying to profit from it;
- Supported and evolved by the vendor, who retains the intellectual property rights;
- Vendor (not acquirer) controls the frequency of the product’s maintenance and updates;
- Available in multiple, identical copies;
- Used without hardware or source code modification.

The promise of COTS is to save development time, reduce technical risk, reduce time-to-market, reduce cost-to-market, and take advantage of latest technology. However, often these promises are not realized. Considerations for COTS-based systems include (Long, 2000):

- COTS products not built to your specific requirements (including missing functionality, extra functionality, and unwanted behaviors).



- Unique, or different than expected, interfaces, including a vendor's use of proprietary data formats and/or communications protocols, may occur.
- Vendor claims and decisions may impact schedule.
- The details needed to understand how COTS products may impact the safe and secure operation of the SoI may not be readily available, including the trustworthiness of the vendor, the use of open-source software, and the use of third-party software of unknown origin.
- COTS product may be insufficiently documented.
- “Not Invented Here” (NIH) syndrome may deter engineers from using COTS.
- Delivery times may not be met.
- Special integration challenges may occur.
- COTS products are often not verified to your specific requirements and may lack verification data for the operating environment for the SoI.
- “Sole Source” suppliers result in more risk.
- Need to consider the entire life cycle cost (LCC) of maintenance and technology rolls/refresh due to COTS obsolescence and diminishing manufacturing sources and material shortages (DMSMS) (Note: IEC 62402 (2019) provides guidance for establishing a framework for obsolescence management process which is applicable through all stages of system life cycle).

There are differences in approaches to SE for COTS-based systems development. Some of the key COTS-based SE considerations are shown in [Table 4.2](#). Effective use of COTS generally requires COTS evaluation starting during needs analysis. In some circumstances, an “internal sales pitch” for each viable candidate COTS-based system needs to be developed, highlighting which requirements are met, which are partially or not met, and what additional capabilities and cost advantages (now and potentially in the future) each possible system provides. For an organization which has only done “make” system development in the past, moving to a COTS-based development requires a different mind-set and different development skills, including the new role of COTS integrator. These skills are often different than those needed for non-COTS-based system development.

**TABLE 4.2** Considerations for COTS-based development efforts

| Aspect                                | Traditional Systems Engineering                                                            | COTS-Based Considerations                                                                                                                                                                                                     |
|---------------------------------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Focus                                 | The SoI                                                                                    | The SoI as well as how potential COTS products in the marketplace could be assembled to meet most/all the needs                                                                                                               |
| Stakeholder Needs and Requirements    | Fairly explicit stakeholder requirements                                                   | Flexible and prioritized capabilities stated in broad terms                                                                                                                                                                   |
| System Requirements and Functionality | Requirements and functionality are defined and allocated based on technical considerations | COTS capabilities and functionality form the basis for the system requirements allocation and evolution and COTS may introduce additional system-level constraints                                                            |
| System Element Requirements           | Extra or missing system element requirements are typically bad                             | Need to strike a balance between what the system needs and what the market can provide, missing or extra COTS requirements may be a reality due to the marketplace and they may also necessitate extra COTS wrappers and glue |